Laugh Track

SDD

Presented to: Mariam El Mezouar and Jonas Zhou

CISC/CMPE 320

Group 23: Absolute Comedy

Doug Byers

Oliver Cao

Brie Chabi

Amanda Goertz

Richard Osborne

Navid Sarshar

Yichen Zou

2021-11-08

1.0 - Purpose of System

The purpose of Laugh Track is to entertain kids ages 8-12 and game enthusiasts. The system is a platformer style game that requires the user to make split second decisions to avoid enemies and collect buffs. With each run the user attempts to get a new high score allowing them to improve their skills and understanding of the game as they continue playing.

The game will implement an infinite level system where the difficulty will progress as the level goes on and the user's score increases. This system will allow for a new experience on every new run without a major increase in the scope of the game. The infinite level system is an alternative to a traditional level system where the player attempts to beat each individual level, with the levels progressively becoming more difficult as the game goes on.

Another purpose of the game is to represent comedy in the form of a game. This will provide a new form of experiencing comedy for comedy fans along with generating new comedy fans. As with most games this will be a purely recreational product that serves few, if any other purpose. To properly complete these objectives the design will emphasis performance, dependability and user friendliness. In addition, safely handing user data is necessary to provide a positive game experience.

2.0 - Use Cases

Use Case Name:

- A user playing the game for entertainment

Participating Actors:

- The player-controlled character, MainCharacter

Flow of Events:

- The player launches the game and is met with the main menu
    - The player can then choose one of the three levels, check high scores, or exit the game
- With a level selected, the player is asked for three letters to represent them if their score makes it to the leaderboard
- The level begins with MainCharacter in a 2D world with the screen slowly moving to the right with slowly increasing speed
- The player can pause at any time where they can resume, quit to main menu, or quit to desktop
- MainCharacter will encounter obstacles and enemies with increasing frequency as they go
- MainCharacter can move in any direction (and jump) as long as the moving screen doesn't catch them
- MainCharacter's score and life bar are displayed at the top of the screen
    - The score constantly increases with distance
    - MainCharacter loses life if they are hit with an enemy attack
    - MainCharacter may collect jokes to increase their life bar and/or score
- When the life bar is depleted the game ends and MainCharacter performs their comedy show (a few different ending screens depending on final score)
- The player is sent to the main menu and may either play again or exit the game

Entry Condition:

- User is bored and wants to play a game or is compelled to beat their high score

Exit Condition:

- The player becomes bored of the game or frustrated that they are unable to beat their high score

Quality Requirements:

- A player should never experience any bugs/crashes while playing the game
- Loading times should be short and the game should perform well on average hardware
- The game should be fun

3.0 - Object Models

There is only one actor for this system, the game player.

Table 1: Showing Entity, Control and Boundary object models

Entity	Control (game control logic)	Boundary (game GUI)
• **Enemy:** the AI-controlled enemy • **Main Character**: the player-controlled character • **Map:** game map with collision models and event triggers • **Coordinate:** representing objects coordinates • **Attack:** collection of player and enemy attacks • **Game Mode:** current selection of game mode • **Game control:** representing the state of game, start, end, pause • **Event Trigger:** event object, environmental or triggered by another object • **Player state:** player information, data synchronization state, authentication token	• **Mechanism Controller:** control of game mechanism like physics and interaction with environment • **Interaction Controller:** Control of interactions between objects • **Synchronize Controller:** Control the communication between online server and local client, manage the local data and configuration	• **Guide:** show the game instruction • **Mode selection:** Select different game modes • **Level selection:** selection of different levels/maps • **Exit:** exit the game • **Pause:** pause the game • **Resume:** continue the game • **Register:** register a new player account • **Login:** login to an existing account • **Synchronize:** upload or synchronize local data/configuration/game progress

4.0 - Sequence Diagrams

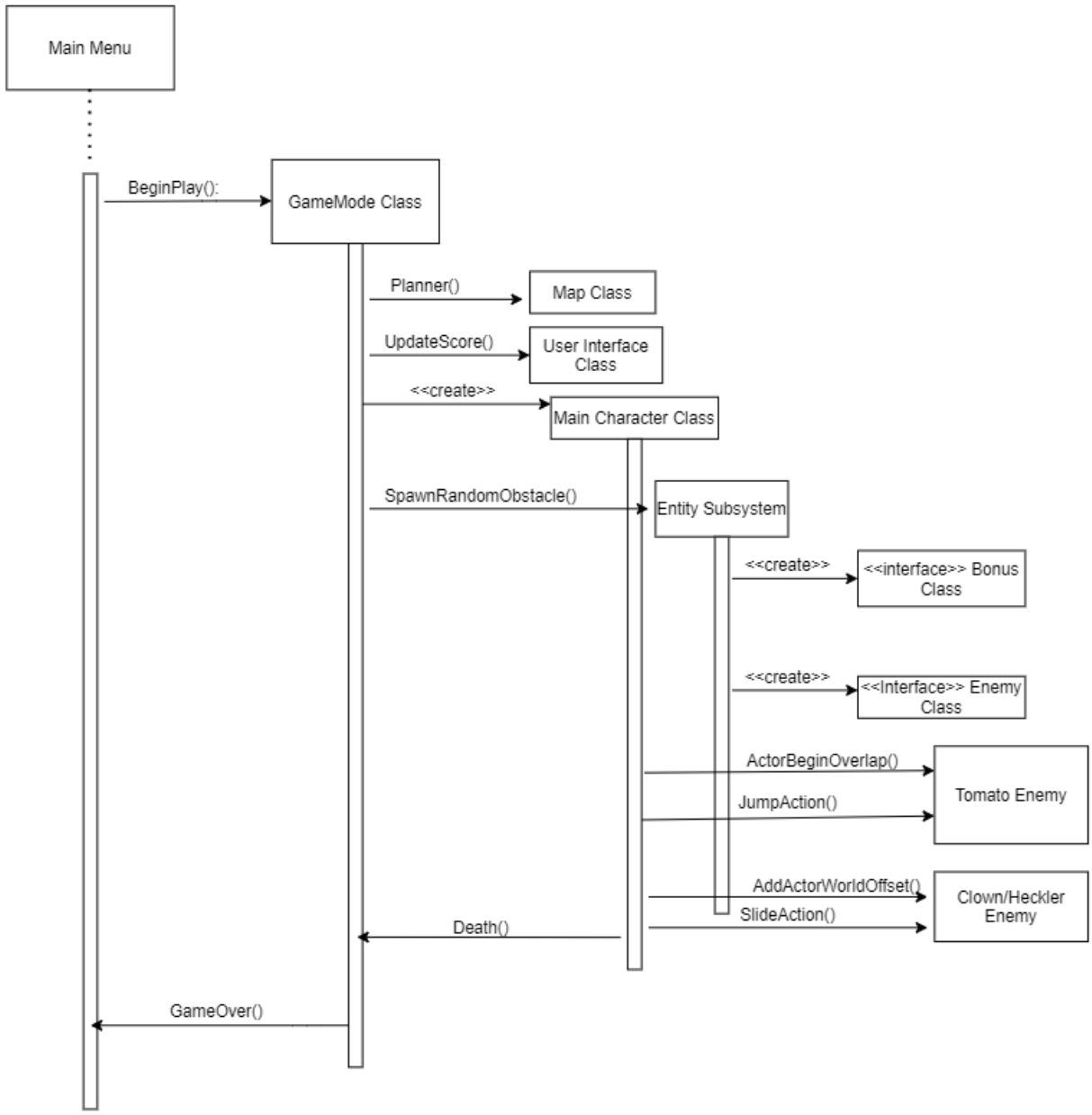

Figure 1: Sequence diagram showing the sequence from the beginning to the end of the game

5.0 - Class Diagrams

5.1 - Diagram

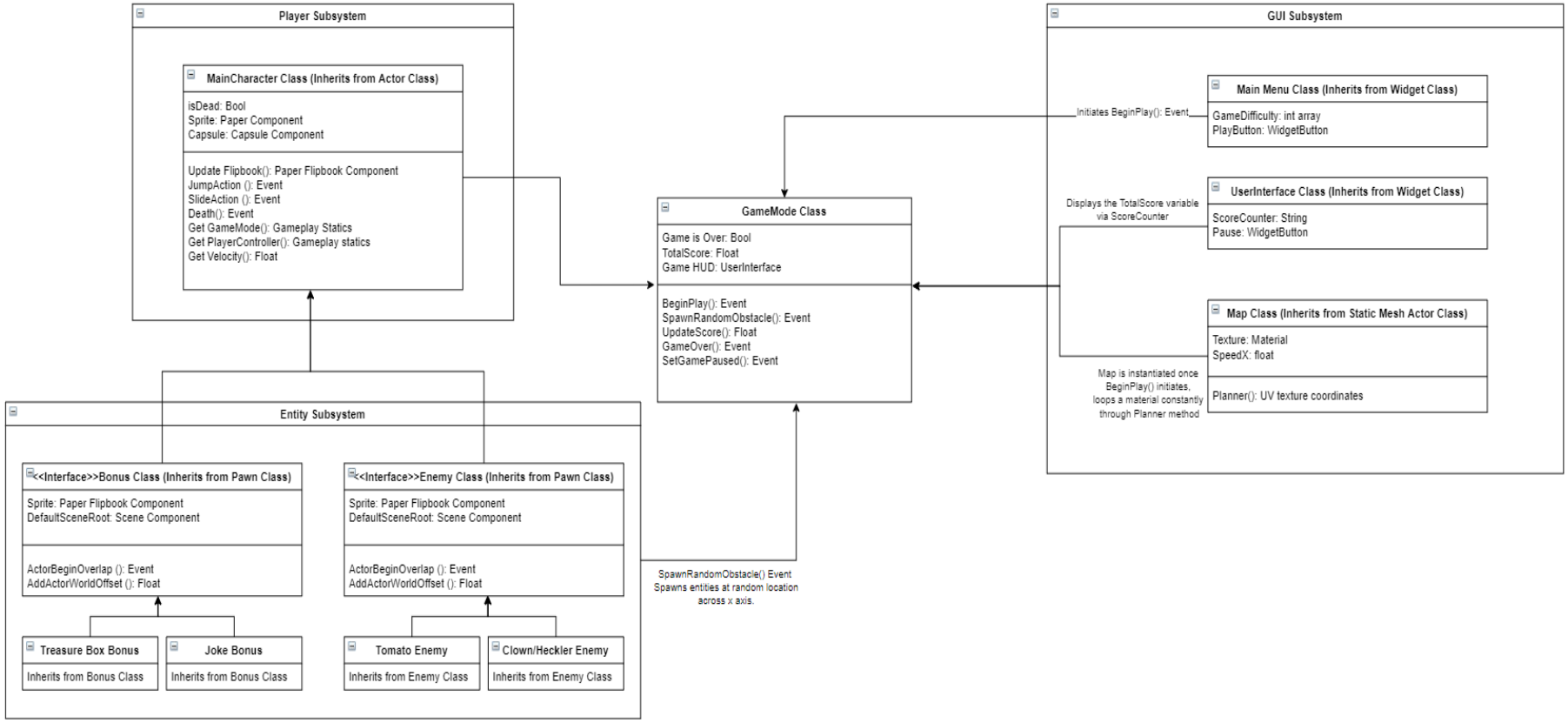

Figure 2: Class diagram explaining classes used in Laugh Track

5.2 Variables and Methods

Since the software is built on Unreal Engine, we have utilized various built-in methods and variable types in the construction of our classes. The various methods and variable definitions can be found below:

Table 2: Describes the software's built in variables and methods

Name	Type	Definition
WidgetButton	Variable	The button is a click-able primitive widget to enable basic interaction, you can place any other widget inside a button to make a more complex and interesting click-able element in your UI.
UV Texture coordinates	Variable	UV texture coordinates are used to create panning, or moving, textures.
set gamePaused()	Method	Event which sets the game's paused state
Scene Component	Variable	A SceneComponent has a transform and supports attachment, but has no rendering or collision capabilities. Useful as a 'dummy' component in the hierarchy to offset others.
Paper Flipbook component	Variable	The PaperFlipbookComponenT is a wrapper for a SourceFlipbook (which is a series of Sprites played sequentially to create a 2D animation). When creating a Paper2DCharacter, this component type is automatically added which allows you to create playable, animated 2D characters.
Paper Component	Variable	A PaperSpriteComponent handles rendering and collision for a single instance of a UPaperSprite asset. This Component is created when you drag a Sprite asset from the Content Browser into a Blueprint or contained inside the Actor created when you drag one into a level.
get PlayerController()	Method	A PlayerController is an interface between the Pawn and the human player controlling it. The PlayerController essentially represents the human player's will.
Gameplay Statics	Variable	Static class with useful gameplay utility functions that can be called from both Blueprint and C++
Event	Variable	Events are nodes that are called from gameplay code to begin the execution of an individual network within the EventGraph. They perform a series of actions in response to certain events that occur within the game, such as when the game starts, when a level resets, or when a player takes damage.
Capsule Component	Variable	A capsule is generally used for simple collision. Bounds are rendered as lines in the editor.
BeginPlay()	Method	Event when play/game begins.
Addactorworldoffset()	Method	Adds a delta to the location of this actor in world space.
Actorbeginoverlap()	Method	Called when another actor begins to overlap this actor, for example a player walking into a trigger. For events when objects have a blocking collision, for example a player hitting a wall, see 'Hit' events.

6.0 - State chart diagrams

The below diagram shows the state changes of our game throughout a round:

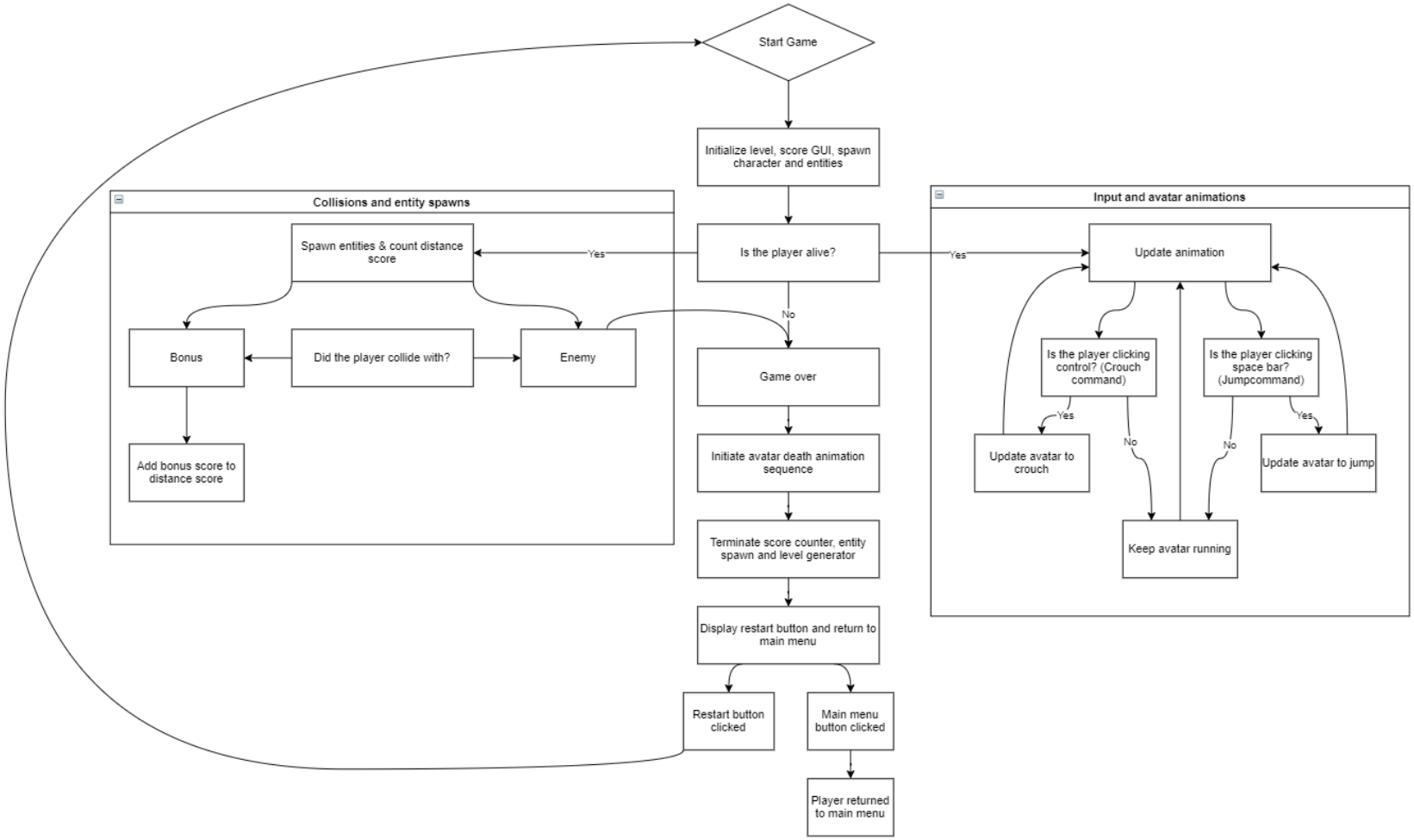

Figure 3: State chart diagram showing different states that could be reached through one round of game play

7.0 - Use case Diagrams

Below is a diagram that shows all use cases:

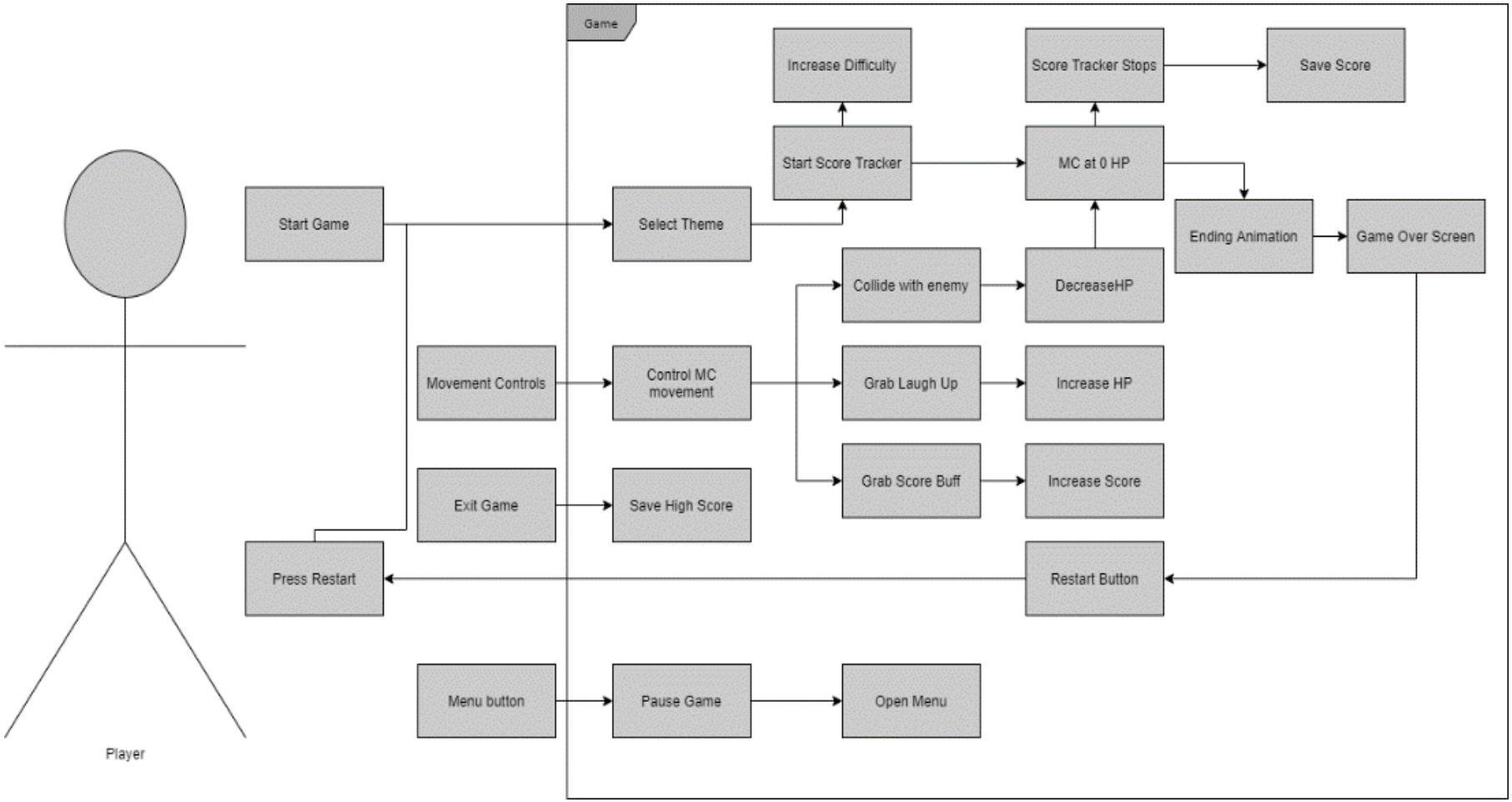

Figure 4: Use case diagram for different actions and controls

8.0 - Design Goals

The 2D platformer game Laugh Track has several embedded functionalities and opportunities for user input.

Table 3: Functionalities of the game during game play

Functionalities	**Description**
Life Bar	Keeping track of how much life the user has left
Tomatoes	Penalty – decreases life bar
Good Reviews	Reward – increases life bar
Distance Tracker	Shows how far you have made it into the game (distance traveled since starting point)
User Input	**Game Command**
Clicking space bar	Jump
Clicking control	Duck
Clicking on the restart/main menu button	Restarts game/changing UI for the user from the game-scape to the main menu

8.1 - Performance

The game has optimal performance. All commands are executed instantaneously after the user input is received.

The game keeps track of the life bar and distance tracker for the entirety of each round, while the response from tomatoes and good reviews is only seen when the user either makes a mistake or collects good reviews along the game's path.

A system requires 1-2 GB of RAM in order to run a 2D Platformer such as Laugh Track.

8.2 - Dependability

The game itself is very dependable and user-friendly. There are only three opportunities for user input, as shown in Table 3.

Due to the lack of ability to make an incorrect input, the game has excellent fault tolerance, safety and security and will never be inoperable due to incorrect user input.

The performance of normal tasks (jumping and ducking) is always available while the game is running. However, if the user makes a request to jump and duck simultaneously, both actions cannot be executed in the same instance. The user must choose which action they want as the game will not perform both.

8.3 - Cost

In order to develop a 2D-platformer game such as Laugh Track, with few functionalities, it could cost anywhere from $5 000 to $60 000.

Since there are few user inputs and functionalities as mentioned in Table 1 and Table 2, a game like Laugh Track would be on the lower end of the price range. However, the cost of promotion and launch can increase the price tag.

Though within the scope of this project, since the game will not be published, the game software Unreal Engine charges no royalties so the cost of the game is free.

8.4 - Maintenance

Increasing the functionality of the game is straightforward. For example, if a feature that tracked how many good reviews was to be embedded in the game, a new unreal engine class could be made that kept a count each good review collected along the game path. Once this class has been created, it can be displayed on the user interface as a new functionality of the game.

If another person with game development background were to look at the Unreal Engine blueprints of the game, they would easily be able to understand how the classes and code is layed out. After reviewing these blueprints, they would be able to add new functionalities to the game as if they were working on the game for the entire project duration.

As for launching this system on different platforms, Laugh Track has been made as a computer application only, therefore it cannot be played on phones, iPod's or tablets.

8.5 - Trade-Offs

Though the game is user-friendly and dependable, it is very simple. There are few functionalities that make it cheaper and more straightforward to create, however, the game is quite repetitive and could get boring for users after a long period of time, making the investment of long-term players quite low. Overall, the game has many excellent attributes from a development perspective as it is simple and little protection needs to be put in the classes to increase security. However, this comes at the cost of the game being not overly fascinating for users for a prolonged period of time.

9.0 - System architecture

9.1 - Software/hardware mapping

The hardware and software used for our game are shown below in the diagram.

Hardware:

- Space bar (Jump): When pressed, the character will perform a running jump. Useful for dodging monsters, jumping over obstacles and avoiding attacks
- Downward Ctrl Key (Sliding): When pressed, the character will perform a sliding animation which can be used as a crouch to avoid monsters.
- Left Click: Used to Select Menu options and restarting the game after losing the game as well as selecting game difficulty.

Figure 5: Diagram showing where user is able to include user input

9.2 - Persistent data management

As Laugh Track is a single player game that is procedurally generated there are few cases where persistent data needs to be kept. However, there is potential for several implementations if there is enough development time left after the completion of more key features. Storing account information, as well as saving maps could help improve player enjoyment but their implementation costs relative to their benefit means their completion would be a tertiary objective.

The users score will be saved when the user dies, if that is higher than any of the 10 scores already saved or less than 10 scores are recorded then the user's score will be kept. If both conditions are false, then the game will not record the score. When the system wants to read the high scores, it can read from the text file that they are saved on.

While account information could be stored encryption would need to be implemented to protect user information and more investigation into the topic is needed. Using a hash algorithm could be implemented easily, but the benefit to the user is questionable as Laugh Track is not an online game and accounts are only used to share high scores.

Saving maps that were randomly generated could allow users to replay and generated maps that they found enjoyable. This would be a much more challenging task than the two previous cases of persistent data management. At a minimum it would require the game to save where and when enemies so that they could be generated at the same spot. However, this would require saving spawn locations while the game is progressing, which would take space on the ram. Since a user would be choosing to

save a map after they have finished their run, the game must save the spawn locations every time. If a seed is used to generate the map then this process becomes significantly easier as the seed number can be saved at the beginning of the run in the RAM before being saved in persistent memory.

9.3 - Access control and security

The access control is not applicable for the project since there will be only one base user and one way of accessing the system. There are two security objectives in a game project. The first is to protect sensitive information, such as email and passwords. The second is to provide a fair arena, which is to prevent cheating.

Our scope is to develop a local single player game that runs without any internet connection. But user will be able to create an account and save/synchronize their game progress with online servers and share their highest score with other players. Referring to the historical penetration tests conducted on unreal-based games, we decided to focus on several problems.

The first is to encrypt the connection to prevent Man-In-The-Middle attacks. The server will be utilizing the DTLS encryption method, to put on an encryption layer over the connection between the client and the data server. This will prevent unauthorized access to information like authentication tokens. Second, during the game development, it will be set to never requiring for administrator privileges to prevent unexpected file modifications. We will also be using binary file analyzing tools to check if unsafe APIs are used.

To prevent cheating is almost impossible since the game is running fully on the local machine, and there is no communication between the online server and local client during the gameplay. Therefore, we cannot tell if the player has done any illegal modification to the data or unauthorized function call. But we can still implement simple anti-cheating by monitoring the memory access and always synchronizing the local data and configuration files. If any unintended illegal memory access is detected, it will be regarded as attempting to cheat and the current game session will be flagged as invalid. Resulting in an invalid highest score and will not be recorded.

9.4 - Global software control

The overall flow of the game will be controlled by the player's input from the mouse and keyboard clicks. The requests will be initiated from the user clicks and the software will produce a result accordingly from the program design.

Starting at the main menu, the left clicks from the mouse will synchronize multiple menu settings switching what is displayed to the player based on their button selection choice. This follows the same menu process request initialization and subsystem synchronization that all menus share, that is only taking action from left clicks on menu button options from the program.

Regarding actual gameplay, the running sequence of the game will respond to keyboard inputs from the user, which are the space bar for jumping and the ctrl key for sliding. When the user interacts with these two keys, it will cause the initiation of requests for jumping and sliding while the character is running. These requests will then trigger a response from the MainCharacter Class and generate action on the screen and cause events that will change the information stored in the Player subsystem attributes.

9.5 - Boundary conditions

9.5.1 - Start-up

When the game is launched, presumably from a .exe file, a window will open and after a short loading time the user will be met with a main menu. From here the player may play the game with the flow of events described in section 1.0: Use Cases. Instead, they may also decided to shut down or just simply check the leaderboards. These cases are described below.

9.5.2 - Shut down

The game can be closed from the main menu, the pause menu, or by simply closing the window. The only data that is saved upon closure are the high scores and they are saved quickly after each run (though may need to investigate whether a shut down at this exact moment would cause problems). There is no need to support saving and loading games because each run will be relatively short and nothing carries over between runs. To avoid accidental loss of progress, when the "Quit to desktop" option is chosen from the pause menu there will be a reminder that progress will not be saved

9.5.3 - Checking Leaderboard

Since the leaderboard is accessible from the main menu, it is possible that a player may check their high scores without playing the game. This may be done to show off a score or to check if a score has been beaten on the local machine. In this case they would launch the game, select leaderboard, and then click back to return to the main menu, where they can then quit to desktop.

9.5.4 - Level Generation

When the player first loads into one of the three levels, the objects within the level must be generated. In order to avoid loading an arbitrarily large level into memory (longer load time and higher RAM usage), a random seed will be generated. The locations of enemies, objects, and jokes will be distributed randomly throughout the level as the player progresses, based on this random seed. This process is done behind the scenes with built-in Unreal Engine functions.

9.5.5 - Exceptions and Bugs

The team expects that the game will have few bugs or exceptions since the inputs by the user are not too complex and the game will be tested rigorously. However, if a crash was to occur, due to a bug in unreal engine, a problem with the user's hardware, or otherwise, there will be an error message. Fortunately, only the progress of the current game will be lost and there should be no issues restarting the game and playing again.

10.0 Subsystem decomposition and subsystem services

The subsystems are broken down into three main subsystems which broken down into smaller subsystems. The three main subsystems include the GUI, Player and Entity subsystems. Each main subsystem besides the Player subsystem will contain a level of abstraction. The GUI subsystem will be composed of three subsystems the menu subsystem, user interface subsystem and level subsystem. Finally, the entity subsystem will contain the bonus subsystem and the enemies subsystem.

10.1 Enemies subsystem

The Enemies subsystem is an element of the entity subsystem and will consist of two main classes/subsystems the tomato and the clown. Both classes share the following services collision with player, movement, and random generation within the level. The only difference between the two classes will be the enemy size/pixel art, movement animation and the position of the enemy with the tomatoes flying and clowns running on the ground. Both the tomato and the clown will serve as subsystems to the enemy subsystem down one level of abstraction. Having only minor and mainly visual

differences between the between both the tomato and the clown allows for strong cohesion within the enemy subsystem as most of the important functional methods are within the subsystem. Very little coupling is required between with the enemy subsystem as most of the main functionality is handled within the subsystem such as interactions and movement.

10.2 Bonus subsystem

The second subsystem is the bonus subsystem which will give the player a score boost once they come in contact with the item. The subsystem will contain two classes the treasure box class and the joke class. This subsystem will offer the following services: interaction with the player and random generation. The overall functionality of the subsystem can be achieved through the use of internal methods with the only interaction required from another subsystem is from the player subsystem to when the objects have collided. With functionality being achieved using internal methods the subsystem has strong cohesion. Since only one interaction occurs the subsystem has weak coupling.

10.3 Player subsystem

The next subsystem is the player subsystem which consists of one main class and the is the main character. The main character will be controlled by the user's keyboard input. This subsystem will provide the following services: receiving input from the user for jumping and sliding, automated character movement (automatically running forward), animations and interactions with obstacles including the level, user interface and enemies. Having most of the actual movement and event handling within the class will provides a high level of cohesion. The area where the subsystem will struggle to meet the design goals is with coupling since the subsystem handles the main character it will have to interact with the levels, main menu, user interface, enemies, and bonuses. With so many interactions between subsystems the main character subsystem will have strong coupling.

10.4 Levels subsystem

The fourth subsystem is the levels, it consists of one class which will create three sperate objects which will form the different levels of the game. The level subsystem will implement a variety of different methods to achieve the following functionality: spawn in main character, spawn in the enemies, interact with user interface, scroll the level, interact with the main menu and the bonuses. The level subsystem will struggle to meet the design criteria with for having weak coupling as levels like the main character is a central subsystem that requires interaction with most of the other subsystems. However, most of the methods used by the subsystem will be internal to the class so the subsystem will have strong cohesion.

10.5 User interface subsystem

The fifth subsystem is the user the interface which consists of one class the user interface that will display the users score. In order to provide the user with there current score the services that the subsystem provides includes keeping track of the distance the player has traveled, their score and number of lives remaining. Overall, the user interface will meet the design goals with strong cohesion and weak coupling. The strong cohesion is shown through implementation of almost all internal methods to function. The weak coupling is achieved by only having to interact with the levels subsystem to receive the required information.

10.6 Menu subsystem

The finial subsystem is the starting menu which will consists of one class the menu class. The menu class will provide the following services starting the game, exit the game, select a level theme, and select level

difficulty. To execute these tasks the subsystem will have to interact with the level subsystem to start the game. With minimal interaction with different subsystems the menu subsystem will have weak coupling. Since most of the functionality for the subsystem is achieved using multiple different internal methods the overall cohesion of the class is strong.

10.8 Overall Coupling and design goals analysis

In order to understand the overall coupling between subsystems the system must be looked at as a whole. First the layering architecture must be chosen, between open and closed architecture. Due to the simplicity of the game and the simple layout of the subsystems there is only layering of two or three levels. With only a maximum of three levels of abstraction and no need to skip a level of abstraction closed architecture is the best choice. Figure 6: Diagram of all subsystems shows all of the subsystems and lines to show their dependencies in order to demonstrate the overall coupling of the system. The figure also shows the decomposition of each subsystem into their separate subsystems (With the arrows representing a level of abstraction). As seen in the diagram the two main elements that require the most coupling are the levels and the main character subsystems. These two subsystems are critical components to the overall game, so they require a lot of coupling. Only having two critical components with strong coupling allows for all the other subsystems to have weak coupling causing for the entire system to have weak coupling. Except for the two critical subsystem, each subsystem can be changed with very little modification to the other subsystems. Overall due to the weak coupling the subsystem decomposition meets the design goals. The main design goals that apply to subsystem decomposition is to have a game that can be easily updated and modified and a game that has good performance. Since the subsystem decomposition has weak coupling, it will all for easy modification of most subsystems to allow for the game to be easily improved and modified as the users see fit. The final design goal that is impacted by the subsystem decomposition is the overall performance of the game. Subsystem decomposition does not necessarily improve the overall performance of the game however could negatively affect the performance if the appropriate coupling between subsystems is not present. The subsystem decomposition that has been implement allows for the performance to not be sacrificed by maintaining all the necessary coupling between subsystems. In conclusion the subsystem decomposition meets the applicable design goals.

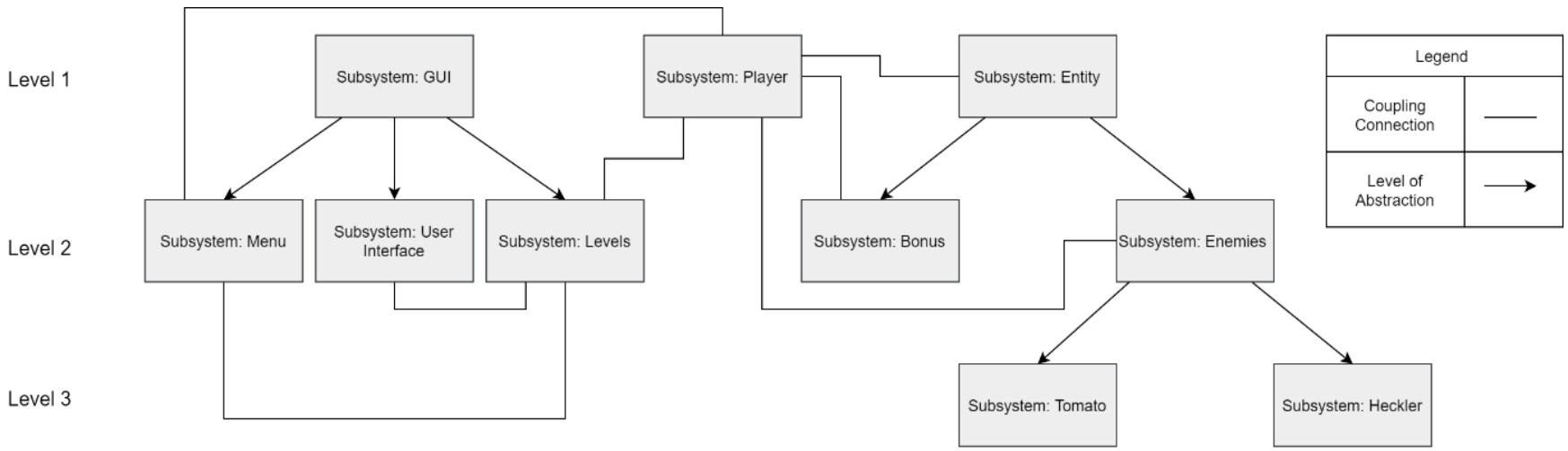

Figure 6: Diagram of all subsystems

11.0 – Definitions and Acronyms and Abbreviations

Laugh-Up: Every time you collect a joke, it increases your health
Critic: A monster that lowers your health when it touches you.
Platformer: A 2D video game
Endless runner: A running game that never ends until you die or run out of laughs (health)
2D graphics: Graphic rendering technique in a two-dimensional game, often using sprites
Tomato'd: A special attack when the audience (monster) does not find you funny and throws a tomato at you.
Joke: The runners self-defense/attack towards monsters after picking up power ups.
Aggro: (aggravation) monsters that will attack you as you near them.
Auto-run: The runner will automatically run without user input after pressing 1 button
Buff: an effect placed on the runner (comedian) that beneficially increases you, whether it is an attack power up or defense powerup.
Main Character: The Main Character is the user controlled entity in this game with an associated life bar.
Enemy: An enemy is a class that represents the antagonists of this game, who will attempt to harm the Main Character.
Life Bar: The Life Bar is a representation of the Main Characters Health. It is affected by colliding with enemies or picking up a Laugh-Up.
Blueprint: A blueprint is a virtual scripting system inside the Unreal Engine, and is being used to prototype this game.
HP: Stands for health points which is a representation for the health of the main character, it will not be visible to the user but will be represented by the Life Bar.
Avatar: A graphical illustration that represents the computer user.
DTLS: Datagram transport layer security is a method of encryption used to prevent eavesdropping, tampering or forgery.
API: An Acronym Programming Interface is a software intermediary that allows two applications to talk to each other.
Leaderboards: A list of high scores that considers multiple users
Seed: A number used to initialize a random generator
Man-In-The-Middle-Attack: A cyber attack that aims to circumvent mutual authentication

11.0 - Screenshots of GUI Interfaces

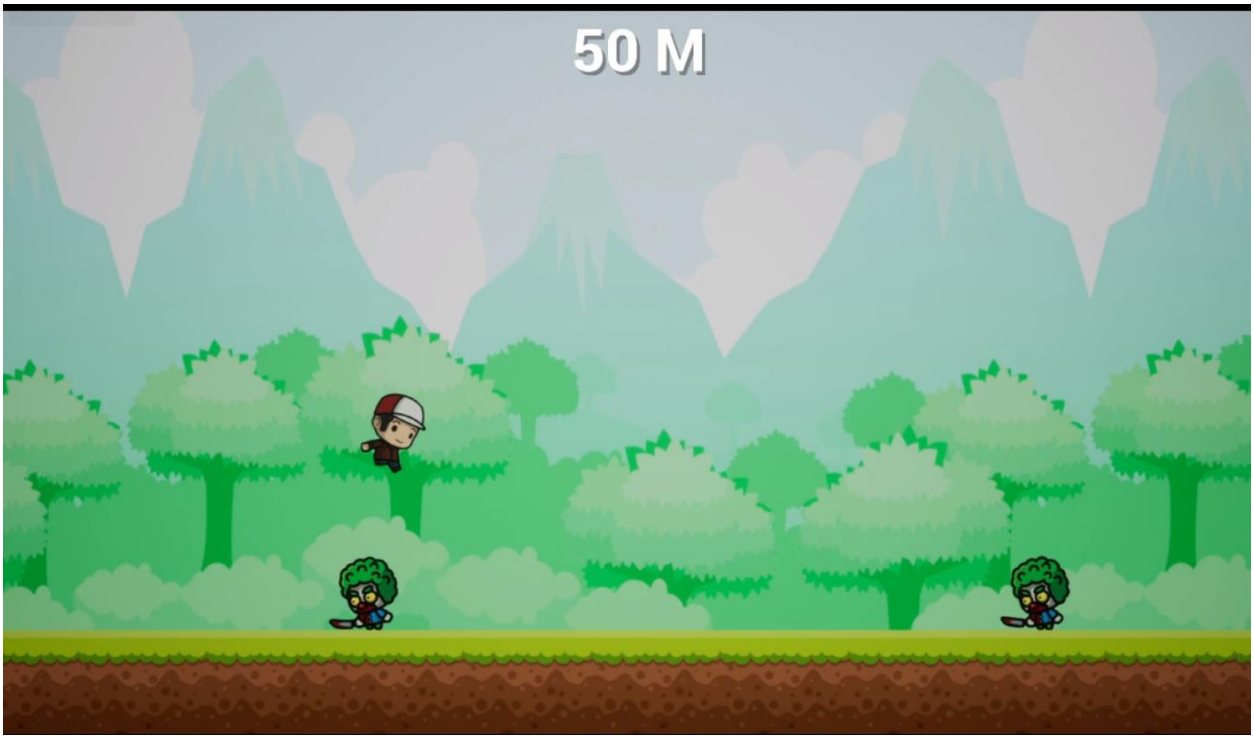

Figure 7: GUI interface of MainCharacter jumping

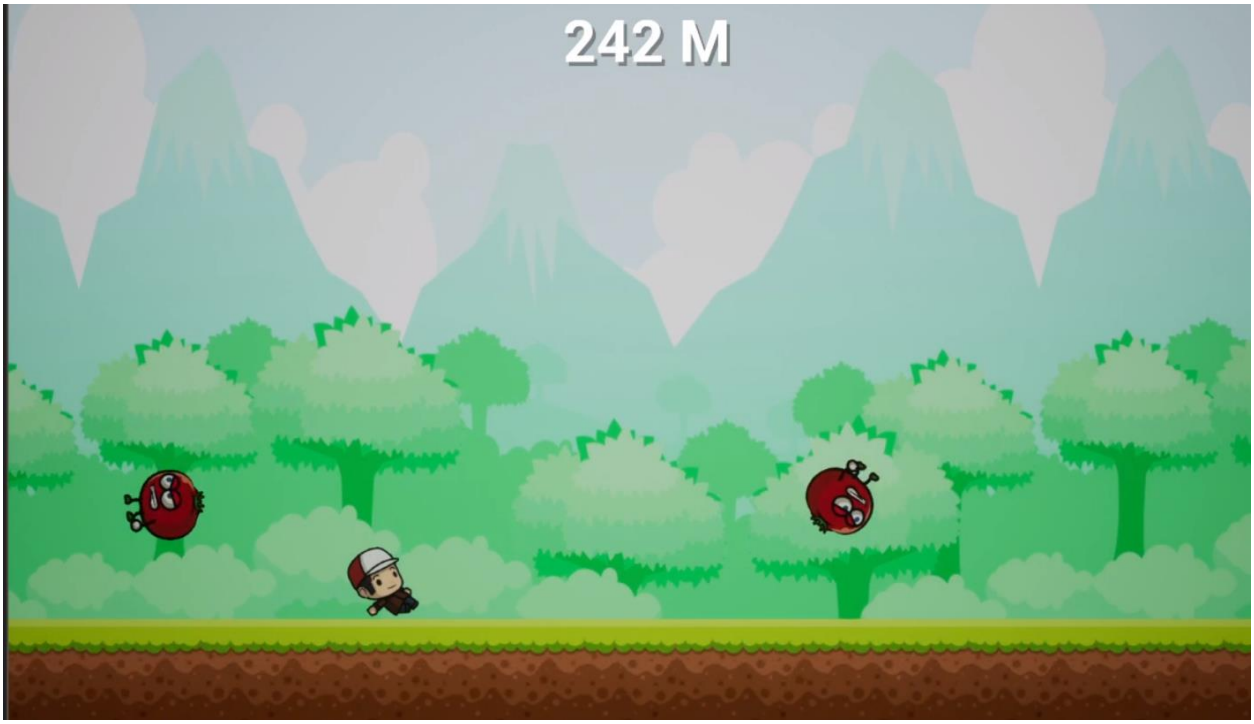

Figure 8: GUI interface of MainCharacter sliding

12.0 - Coding Assignments

Table 4: Coding assignments for the game

Brie Chabi	GameMode class	Game state	Ensure the proper handling of the game states throughout the software.
Amanda Goertz	MainCharacter class	Player	Ensure the player controls, animations and collisions are working as intended throughout the level.
Doug Byers	UserInterface class	GUI	Ensure the proper information such as score tracking and functionality of pause screen are displayed.
Chen Zou	MainMenu class	GUI	Ensure that the implemtation of a game main menu is working correctly such as selecting difficulty level.
Navid Sarshar	Map class	GUI	Ensure the implementation of the map such as the background and speed is working properly.
Richard Osborne	Bonus class	Entity	Ensure that the player can collide with bonus items and gain extra score as a result.
Oliver Cao	Enemy class	Entity	Ensure that the player can collide with enemy obstacles and start death sequence, terminate score countring.

13.0 - Timeline

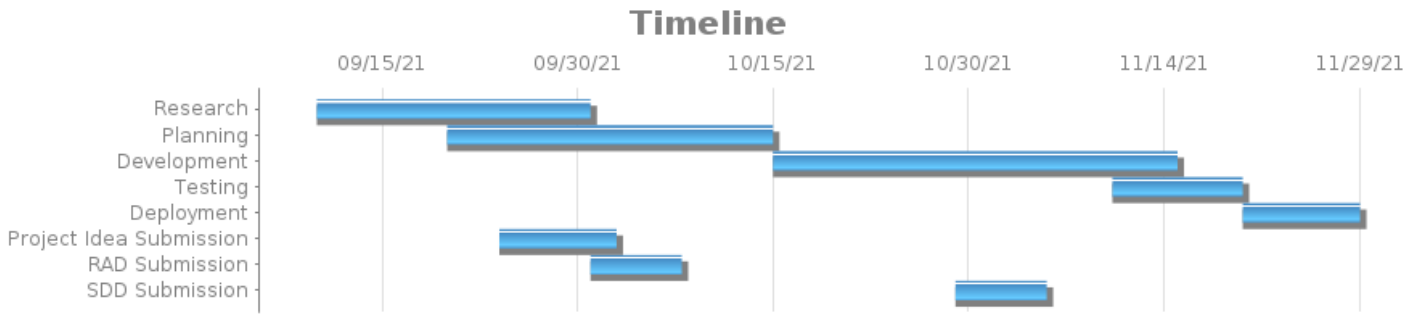

Figure 9: Timeline from beginning to end of the project